

S-13

```
import pandas as pd
df=pd.read_csv('/content/amazon_products_sales_data_cleaned.csv', engine='python')
df.head()
```

	product_title	product_rating	total_reviews	purchased_last_month	discounted_price	original_price	is_best_seller	is_sponsor
0	BOYA BOYALINK 2 Wireless Lavalier Microphone f...	4.6	375.0	300.0	89.68	159.00	No Badge	Sponsor
1	LISEN USB C to Lightning Cable, 240W 4 in 1 Ch...	4.3	2457.0	6000.0	9.99	15.99	No Badge	Sponsor
2	DJI Mic 2 (2 TX + 1 RX + Charging Case), Wirel...	4.6	3044.0	2000.0	314.00	349.00	No Badge	Sponsor
3	Apple AirPods Pro 2 Wireless Earbuds, Active N...	4.6	35882.0	10000.0	162.24	162.24	Best Seller	
4	Apple AirTag 4 Pack. Keep Track of and find Yo...	4.8	28988.0	10000.0	72.74	72.74	No Badge	Organ

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
import numpy as np
import statistics as stats
import pandas as pd

# Example IQ data
iq_scores = [85, 90, 95, 100, 100, 105, 110, 115, 120, 125, 130]

# --- Measures of Central Tendency ---
mean_iq = np.mean(iq_scores)
median_iq = np.median(iq_scores)
mode_iq = stats.mode(iq_scores)

# --- Measures of Dispersion (Variation) ---
std_dev = np.std(iq_scores, ddof=1) # sample standard deviation
variance = np.var(iq_scores, ddof=1)
iq_range = np.ptp(iq_scores) # range (max - min)

# --- Quartiles and IQR ---
q1 = np.percentile(iq_scores, 25)
q3 = np.percentile(iq_scores, 75)
iqr = q3 - q1

# --- Display Results ---
print("📊 IQ Statistics Summary")
print(f"Mean IQ: {mean_iq:.2f}")
print(f"Median IQ: {median_iq}")
print(f"Mode IQ: {mode_iq}")
print(f"Standard Deviation: {std_dev:.2f}")
print(f"Variance: {variance:.2f}")
print(f"Range: {iq_range}")
print(f"Q1 (25th percentile): {q1}")
print(f"Q3 (75th percentile): {q3}")
print(f"IQR (Interquartile Range): {iqr}")
```

```
📊 IQ Statistics Summary
Mean IQ: 106.82
Median IQ: 105.0
Mode IQ: 100
Standard Deviation: 14.54
Variance: 211.36
Range: 45
Q1 (25th percentile): 97.5
```

```
Q3 (75th percentile): 117.5
IQR (Interquartile Range): 20.0
```

```
#Machine(computer/device) learning(learn from the data)
#machine learning = new born baby's mind

#types of machine learning
#supervised ML (LABELLED)
#unsupervised ML (UNLABELLED)
#reinforcement ML (BOTH)
```

```
#SUPERVISED ML
# Regression (output/prediction- number(age,salary,price..))
# Classification (output/predictions- class(pass/fail, yes/no))
```

```
#regression - (output - number/continuous)
#Linear regression y=mx+c
```

```
# Import libraries
import numpy as np #numerical
import matplotlib.pyplot as plt #visualization
from sklearn.linear_model import LinearRegression

# Example Data
# Let's say we're studying the relationship between hours studied (X) and marks obtained (Y)
X = np.array([1, 2, 3, 4, 5, 6]).reshape(-1, 1) # independent variable (hours)
Y = np.array([35, 40, 50, 55, 65, 70]) # dependent variable (marks)

# Create a linear regression model
model = LinearRegression() #calling

# Fit the model to the data
model.fit(X, Y) #training the data

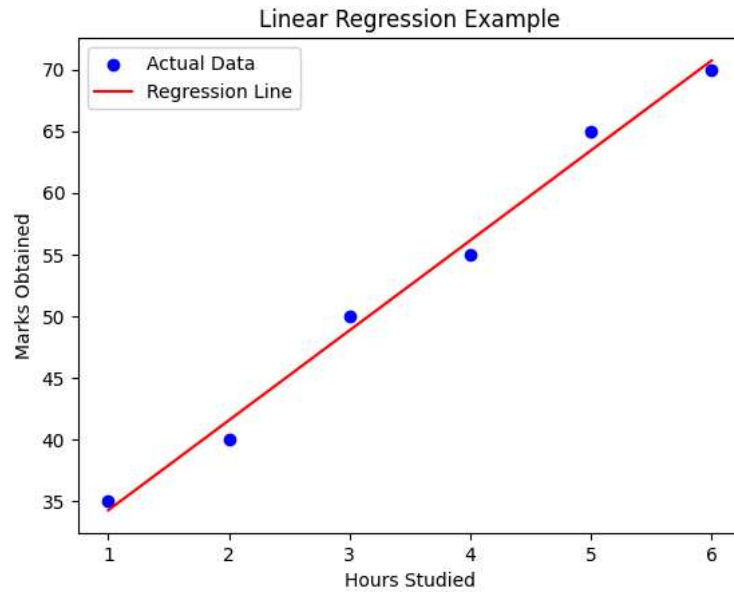
# Predict marks for the given hours
Y_pred = model.predict(X)

# Print results
print("📊 Linear Regression Results:")
print(f"Coefficient (Slope): {model.coef_[0]:.2f}")
print(f"Intercept: {model.intercept_:.2f}")
print(f"Equation of Line: Y = {model.coef_[0]:.2f}X + {model.intercept_:.2f}")

# Plot the data and regression line
plt.scatter(X, Y, color='blue', label='Actual Data')
plt.plot(X, Y_pred, color='red', label='Regression Line')
plt.xlabel('Hours Studied')
plt.ylabel('Marks Obtained')
plt.title('Linear Regression Example')
plt.legend()
plt.show()
```



Linear Regression Results:
 Coefficient (Slope): 7.29
 Intercept: 27.00
 Equation of Line: $Y = 7.29X + 27.00$



```
#metrics of linear regression
#logistic regression
#metrics of logistic regression
```

```
import pandas as pd
df = pd.read_csv('/content/Salary_dataset.csv', index_col=0)
display(df.head())
```

	YearsExperience	Salary	
0	1.2	39344.0	
1	1.4	46206.0	
2	1.6	37732.0	
3	2.1	43526.0	
4	2.3	39892.0	

```
# This cell is no longer needed as the 'Unnamed: 0' column was handled during data loading.
# df = df.drop('Unnamed: 0', axis = 1)
```

```
df.head(
)
```

	YearsExperience	Salary	
0	1.2	39344.0	
1	1.4	46206.0	
2	1.6	37732.0	
3	2.1	43526.0	
4	2.3	39892.0	

```
df.shape
```

```
(30, 2)
```

```
df.describe()
```

	YearsExperience	Salary	
count	30.000000	30.000000	
mean	5.413333	76004.000000	
std	2.837888	27414.429785	
min	1.200000	37732.000000	
25%	3.300000	56721.750000	
50%	4.800000	65238.000000	
75%	7.800000	100545.750000	
max	10.600000	122392.000000	

```
import seaborn as sns
#1 scatter plot
plt.figure(figsize=(8,6))
sns.scatterplot(x='YearsExperience', y='Salary', data=df, color='blue', s=70)
plt.title('Years of Experience vs Salary', fontsize=14)
plt.xlabel('Years of Experience')
plt.ylabel('Salary (in ₹)')
plt.grid(True)
plt.show()

# --- 3 Regression Line (Trend) ---
plt.figure(figsize=(8,6))
sns.regplot(x='YearsExperience', y='Salary', data=df, color='red', line_kws={"color":"black"})
plt.title('Salary Trend with Experience', fontsize=14)
plt.xlabel('Years of Experience')
plt.ylabel('Salary (in ₹)')
plt.show()

# --- 4 Distribution of Salary ---
plt.figure(figsize=(8,5))
sns.histplot(df['Salary'], kde=True, color='purple')
plt.title('Salary Distribution')
plt.xlabel('Salary')
plt.ylabel('Frequency')
plt.show()

# --- 5 Boxplot to Detect Outliers ---
plt.figure(figsize=(6,4))
sns.boxplot(y='Salary', data=df, color='skyblue')
plt.title('Salary Spread and Outliers')
plt.ylabel('Salary (in ₹)')
plt.show()
```







```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# --- Split into Features (X) and Target (Y) ---
X = df[['YearsExperience']]
Y = df['Salary']

# --- Train-Test Split ---
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)

# --- Create and Train the Model ---
model = LinearRegression()
model.fit(X_train, Y_train)

# --- Predictions ---
Y_pred = model.predict(X_test)

# --- Model Parameters ---
slope = model.coef_[0]
intercept = model.intercept_

# --- Metrics ---
r2 = r2_score(Y_test, Y_pred)
mse = mean_squared_error(Y_test, Y_pred)
rmse = np.sqrt(mse)

# --- Results ---
print("📊 Linear Regression Model Summary")
print(f"Coefficient (Slope): {slope:.2f}")
print(f"Intercept: {intercept:.2f}")
print(f"Equation of Line: Salary = {slope:.2f} × Experience + {intercept:.2f}\n")

print("📈 Model Performance Metrics")
print(f"R² Score (Accuracy): {r2:.4f}")
print(f"Mean Squared Error: {mse:.2f}")
print(f"Root Mean Squared Error: {rmse:.2f}")

# --- Visualization ---
plt.figure(figsize=(8,6))
sns.scatterplot(x=X_test['YearsExperience'], y=Y_test, color='blue', label='Actual Salary')
sns.lineplot(x=X_test['YearsExperience'], y=Y_pred, color='red', label='Predicted Salary')
plt.title('Linear Regression: Experience vs Salary')
plt.xlabel('Years of Experience')
plt.ylabel('Salary (in ₹)')
plt.legend()
plt.show()
```



Linear Regression Model Summary

Coefficient (Slope): 9423.82

Intercept: 24380.20

Equation of Line: Salary = 9423.82 × Experience + 24380.20

Model Performance Metrics

R² Score (Accuracy): 0.9024

Mean Squared Error: 49830096.86

Root Mean Squared Error: 7059.04



```
#calculate key metrics like:
"""
Coefficient (slope)

Intercept

R2 score (accuracy)

Mean Squared Error (MSE)

Root Mean Squared Error (RMSE)
"""

'\nCoefficient (slope)\n\nIntercept\n\nR2 score (accuracy)\n\nMean Squared Error (MSE)\n\nRoot Mean Squared Error (RMSE)\n'
```

```
#LOGISTIC REGRESSION- Classification (diabetic/not diabetic, fail/pass, yes/no)
#metrics(evaluation)
#accuracy, precision, f1, recall...
```

```
import pandas as pd
df =pd.read_csv('/content/Social_Network_Ads.csv')
df.head()
```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
df['Purchased'].value_counts()
```

	count
Purchased	
0	257
1	143

```
dtype: int64
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, roc_auc_score, roc_curve
```

```
# Encode Gender (Male = 1, Female = 0)
df['Gender'] = df['Gender'].map({'Male': 1, 'Female': 0})

# Select features and target
X = df[['Age', 'EstimatedSalary']] # Independent variables
y = df['Purchased']                # Dependent variable

# Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Feature Scaling
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

# Build Logistic Regression model
model = LogisticRegression(random_state=42)
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

# Evaluate Model
print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("\nROC-AUC Score:", roc_auc_score(y_test, y_prob))

# Plot ROC Curve
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
plt.plot(fpr, tpr, color='blue', label='ROC curve (area = %0.2f)' % roc_auc_score(y_test, y_prob))
plt.plot([0, 1], [0, 1], color='red', linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC Curve)')
plt.legend()
plt.show()

# Example predictions
sample = np.array([[30, 87000]]) # Example: Age=30, Salary=87000
sample_scaled = sc.transform(sample)
print("\nPrediction for Age=30, Salary=87000:", model.predict(sample_scaled)[0])
```

